LKlones: A Dataset of Linux Kernel Mailing List Discussions on Code Duplication

Natália Salvino André
University of São Paulo (IME-USP)
São Paulo, Brazil
natalia.andre@ime.usp.br

Guilherme Ivo Bozi
University of São Paulo (IME-USP)
São Paulo, Brazil
guilherme.bozi@usp.br

Gabriel Almeida
University of São Paulo (ICMC-USP)
São Paulo, Brazil
gabrielsousa230@usp.br

David Tadokoro
University of São Paulo (IME-USP)
São Paulo, Brazil
davidbtadokoro@ime.usp.br

Paulo Meirelles
University of São Paulo (IME-USP)
São Paulo, Brazil
paulormm@ime.usp.br

***Abstract*—Code duplication is one of the best-known forms of technical debt. It makes maintenance more laborious and can lead to inconsistencies, such as a bug being fixed in one copy but not in the others. In a large, long-lived project like the Linux kernel, decisions about duplicated code are debated in the open on mailing lists, where developers propose, question, and reject refactorings, shared helpers, and copied implementations. These discussions are a rich source for understanding how this debt is handled in practice, but they are buried among millions of unrelated emails. We present LKlones, a dataset of Linux kernel mailing list threads about code duplication, built on top of the LKML5Ws dataset. Our pipeline groups emails into threads, in which a regex filter first selects candidate threads and an AI-based classifier sorts each candidate into categories. Out of 3.7 million threads from 206 mailing lists, the classifier labels more than 186,000 threads as related to code duplication, and over 33,000 of them are linked to commits that reached the mainline kernel. Each thread includes its messages, its category with a confidence score, and its links to commits.**

***Index Terms*—Linux kernel, Code Duplication, Mailing Lists**

I. Introduction

Code duplication is one of the best-known forms of technical debt, being the first entry in Fowler's catalog of code smells [1]. This debt can be observed in large, high-impact software projects, such as the Linux kernel, a widely known project maintained for over three decades by a large community. Despite its careful architecture, it still contains duplicated code, including large clones of files and directories [2]. Since its code and development history are publicly available in large volumes, exploring how the kernel addresses this form of technical debt can offer valuable insights into how developers manage this debt in practice.

Prior work has explored this history only partially. Wang and Godfrey searched the Linux commit history for the phrase *"duplicated code"* to build a dataset of intentional clone refactoring [3]. Their search relies on a single phrase and sees only merged changes, missing the rejected ones and the review discussions around the topic. These discussions take place on the kernel mailing lists [4], making them a natural source for closing this gap. Mining them, however, is not straightforward, since duplication is rarely mentioned in developer communication. In commit messages, for instance, only 348 of the 28,538 analyzed by Wang and Godfrey mention duplicated code [3]. Finding the relevant threads among millions of emails therefore requires a systematic selection process.

In this paper, we present LKlones, a dataset of Linux kernel mailing list threads on code duplication. It builds on LKML5Ws [5], a dataset with almost 30 million emails exchanged on the Linux kernel mailing lists. A pipeline builds the dataset, starting by reconstructing all the threads for each list, since a patch discussion involves several emails. It then identifies the threads about duplication by combining a *regular expression* (regex) filter with an AI-based classification. Finally, it links these threads to the commits that reached the mainline kernel.

From 23.8 million emails and 3.7 million threads across 206 mailing lists, we classify 186,432 threads as code-duplication-related, providing the content of each thread along with the full discussion around it. Of these threads, 33,209 are linked to mainline commits, indicating they relate to merged changes. This dataset supports studies of how kernel developers handle duplication, providing complete discussions on the topic, including patches that maintainers rejected or ignored.

II. Background

Code clones. A code clone is a fragment of source code that is identical or similar to another fragment, often resulting from copying and pasting. The literature commonly distinguishes four types of code clones [6]: Type-1 (identical except for whitespace and comments), Type-2 (also allowing changes to identifiers and literals), Type-3 (allowing the addition, removal, or modification of statements), and Type-4 (syntactically different but semantically equivalent). Developers rarely use the terminology established in clone detection research in discussions, preferring expressions such as *duplicate*, *deduplication*, or *copy-paste*, among others.

Mailing lists and patches. The Linux kernel is organized into *subsystems*, each typically associated with a set of maintainers, one or more mailing lists, and a Git repository (*kernel tree*). The `MAINTAINERS` file identifies the maintainers, lists each subsystem's maintainers, files, and mailing

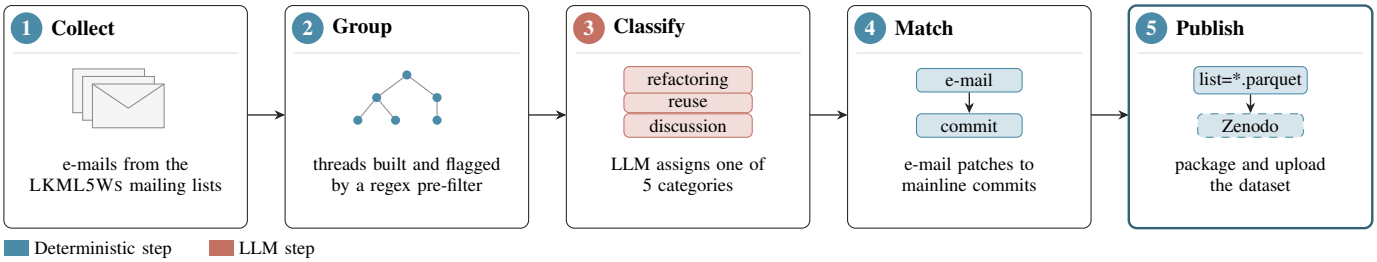


Fig. 1. Overview of the dataset construction pipeline.

lists. *Patches* is the jargon for contributions sent by email to the subsystem's list [7]. Mailing lists remain the primary communication channel, and key maintainers consider them the most suitable solution for such a decentralized project [8]. These discussions are publicly archived in *Kernel Lore* [9], making the development process and its associated discussions accessible for analysis. A patch is analogous to a commit: the subject header is the commit message title, and the body contains the commit message body, the proposed *code* change in a format similar to `git diff`, and *trailers* such as `Signed-off-by`, `Reviewed-by`, and `Link`.

Threads and mainline. Each email has a unique Message-ID, and a reply references its parent through the `In-Reply-To` and `References` headers. The chain of replies forms a *thread*, which gathers the patch, the review comments, and sometimes revised versions of the same patch. Accepted patches become commits in the maintainer's tree, and may later be merged into higher-level trees through pull requests, which are also sent by email (subjects with `[GIT PULL]`), until they reach the *mainline* kernel, the tree from which official releases are made [10]. The kernel documentation recommends that patches carry `Link` trailers pointing to related discussions, preferably on Kernel Lore [7], but this is not mandatory, and its use varies across subsystems. Rejected or ignored patches may never reach the mainline, so their discussions exist only on the mailing lists.

III. Data Collection and Processing

Figure 1 summarizes the five-step pipeline that builds LKLones. Once the threads are assembled, no step removes data; each step only adds columns to the dataset, allowing users to apply their own filters.

A. Data Collection

The input is LKML5Ws [5], a dataset of the mailing lists hosted on Kernel Lore, with one email per row. LKML5Ws also covers lists unrelated to the kernel source code, such as the Git mailing list. To keep only kernel development, we accept a list when it is the official list of a subsystem in the kernel `MAINTAINERS` file that covers repository files in at least one release between v2.6.30 and v7.2. This criterion retains 206 of the 353 LKML5Ws lists in our dataset.

B. Thread Reconstruction

We group the emails into threads with a union-find structure over the `In-Reply-To` header, using the last identifier in `References` when the parent is missing from LKML5Ws. Messages within a thread are kept in chronological order. This is the only step that removes data, as we discard pull requests, which only summarize patches discussed in other threads, and messages sent by bots, such as SYZBOT [11]. The result is 3.7 million threads built from 23.8 million emails.

TABLE I
Parquet dataset schema. Each row represents a thread.

Column	Type	Description
`thread_id`	String	Unique identifier of the thread.
`list`	String	Originating mailing list.
`n_messages`	Int64	Number of messages in the thread.
`message_ids`	List[String]	Message-IDs in chronological order.
`date`	Timestamp	Date of the first message.
`subject`	String	Subject of the first message.
`dup_candidate`	Boolean	Flag for regex pre-filter.
`thread_content`	String	Concatenated content of the messages.
`category`	String	Classification assigned to the thread.
`category_confidence`	Double	Confidence score of the classification.
`commit_matches`	List[Struct]	Commits associated with thread.
Fields of `commit_matches`		
`message_id`	String	Message associated with the commit.
`commit_hash`	String	Hash of the mainline commit.
`status`	String	`LINK`, `EXACT`, or `AMBIGUOUS`.
`ambiguous_ids`	List[String]	Candidate messages for `AMBIGUOUS`.

Since duplication is rarely discussed, we use regex to pre-filter candidate threads. We built the regexes from a vocabulary analysis, in which two reviewers manually inspected a sample of threads and identified the terms that developers use when discussing duplication, such as *duplicat**, *dedup**, *redundant**, *copy-paste*, and phrases like *"move into a common helper"*. The same analysis revealed recurring false positives, such as *"redundant null check"* and *"duplicate include"*, which we added as exclusions. Quoted text is ignored, so that each excerpt only counts in the original message. The pre-filter flags 373,264 candidate threads using the `dup_candidate` column (Table I), and only these receive the concatenated text of their messages, stored in the `thread_content` column.

C. Duplication Classification

The pre-filter favors recall, so many candidates are not actually about duplication. To separate them, a large language model (LLM) determines whether each candidate thread is a duplicate. In preliminary tests, the model used performed

poorly when asked yes-or-no questions directly. We therefore guide it with five categories that describe what we consider related to duplication. A thread is *clone refactoring* when a patch consolidates existing copies into a single place, *preventive reuse* when a patch avoids writing a copy by reusing or sharing code, and *clone and reuse* when both occur in the same thread. It is *duplication discussion* when participants discuss duplication, but no patch acts on it, and *not duplication* otherwise. The categories serve only as guidance: the first four are mapped to *yes*, and the last to *no*.

We use JEV [12], version 1.13.0, selected after comparing it with other LLMs. We frame the task as a choice question, in which JEV picks one of the five categories for each thread. Instead of free-form text, JEV returns the probability for each category, thereby keeping the output structured and reproducible across runs. The chosen category and its probability are stored in the `category` and `category_confidence` columns, respectively, allowing users to set their own confidence threshold. Threads longer than the model context are truncated, keeping the first message and the passages around the pre-filter terms.

To evaluate the classifier, two authors independently labeled 313 threads sampled from four lists, reaching 87.9% agreement (Cohen's $\kappa = 0.82$) and resolving disagreements by consensus. On the binary decision, JEV reaches 84.7% accuracy, with 95.3% recall and 70.3% precision for threads about duplication. The classifier thus rarely misses relevant threads, and users who need higher precision can rely on `category_confidence`. In total, 186,432 threads are classified as related to duplication.

D. Commit Matching

After classifying the threads, we then link them to the commits in the mainline kernel. For each commit, we search for the email that introduced it. Since the `Link` trailer is not present in all commits, we fall back to other criteria. A commit is linked with status `LINK` when its `Link` trailer cites the email Message-ID of an email. Otherwise, it is linked with status `EXACT` when the commit author email and author date match the sender and date of a patch, or with status `AMBIGUOUS` when it matches more than one patch, in which case all candidates are kept. These links are stored in the `commit_matches` column. Overall, 33,209 threads about duplication are linked to 84,964 distinct commits. Of the 196,649 entries in `commit_matches`, 57.5% have status `LINK`, 37.0% `EXACT`, and 5.5% `AMBIGUOUS`.

E. Anonymization and Publication

We use the anonymized release of LKML5WS, in which SHA-1 hashes already replace participants' names and email addresses, but Message-IDs are kept in clear text. Since a Message-ID often embeds the sender's address or host name, we complete this anonymization by applying the same hash to every Message-ID in the output. Each thread is identified by the hash of its first message (`thread_id`). The resulting dataset is stored in Apache Parquet, with one row per thread and the columns described in Table I.

IV. DATASET OVERVIEW

Table II summarizes LKLONES by thread category. Of the 373,264 candidate threads, the classifier assigns 186,432 to the four duplication categories and 186,832 to *not duplication*, so about half of the candidates are false positives, as expected from a filter designed for recall. The classifier is less certain about the threads it assigns to a duplication category (median confidence 0.75, with 27.3% reaching 0.9) than about those it discards (0.95), and least certain about *clone and reuse* (0.46). Threads about duplication have a median of 15 messages, but `n_messages` also counts other patches in a series, so it does not, by itself, indicate that reviewers replied.

TABLE II
THREADS PER CATEGORY[a].

Category	Threads	%	Conf.	Linked (%)
Clone refactoring	71,860	1.9	0.76	24.2
Preventive reuse	56,845	1.5	0.67	16.8
Clone and reuse	1,098	<0.1	0.46	16.1
Duplication discussion	56,629	1.5	0.84	10.7
Duplication-related	186,432	5.0	0.75	17.8
Not duplication	186,832	5.1	0.95	21.8
Not a candidate	3,325,611	89.9	–	–
Total	3,698,875	100.0	–	–

[a] Percentages (%) refer to all threads. Conf. is the median of `category_confidence`. Linked is the share of threads with at least one mainline commit.

V. RELATED DATASETS

Wang and Godfrey [3] built a dataset of intentional clone refactoring in the Linux kernel by searching commit messages for the phrase *"duplicated code"*. Since it starts with the commit history, it covers only merged changes and excludes the review discussions around them. LKLONES starts from the mailing lists instead, so it also includes patches that never reached the mainline and discussions in which no patch acts on duplication, and it selects threads with a broader vocabulary and an LLM classifier rather than a single phrase.

Xu and Zhou [13] collected 666,550 patches sent to the main Linux kernel mailing list (LKML) between 2008 and 2017, together with their review comments, from the kernel Patchwork. They link 139,664 of these patches to mainline commits by matching the commit author time to the email send time, falling back to the commit subject. LKLONES shares the goal of linking discussions to commits, but covers 206 subsystem lists instead of LKML alone, rebuilds complete threads from the email headers instead of relying on Patchwork, and focuses on threads about code duplication.

VI. SHOWCASE

LKLONES can help close gaps in prior studies of code duplication in the Linux kernel. Wang and Godfrey [3] note that little is known about the design decisions that drive

clone refactoring, since commit messages rarely record them. In LKLONES, these decisions can be studied by selecting the threads whose `category` is *clone refactoring* or *clone and reuse*, optionally above a `category_confidence` threshold, and keeping those with more than one message (`n_messages`), that is, those that likely received feedback. The `thread_content` of these threads holds the patch description, where the author justifies the refactoring, and the replies, where reviewers question it, suggest alternatives, or ask for new versions.

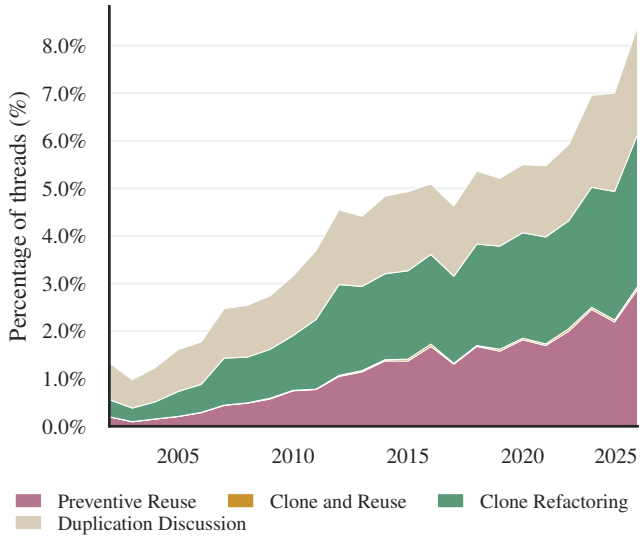


Fig. 2. Yearly proportion of all threads in each duplication-related category (stacked areas).

The dataset also supports other kinds of analysis. For instance, the `date` and `category` columns show how the topic of duplication in the Linux mailing lists has grown over time and how threads are distributed across categories. Figure 2 shows that the share of threads about duplication grew steadily, from about 1% of all threads in 2005 to about 8% in 2025.

An example of a research question that the dataset can help answer is *"Why do some clone refactoring patches never reach the mainline?"* To illustrate this, we use the `linux-iio` threads in LKLONES. We select the 95 threads classified as *clone refactoring* with `category_confidence` ≥ 0.9, keep those with an empty `commit_matches`, that is, those whose patch version has no corresponding mainline commit, and keep those in which someone other than the author of the first message replied. Since the hashed identities are consistent within a thread, the senders can be recovered by parsing `thread_content`. This leaves 55 threads. An LLM (GPT-5.4 nano) then reads the `thread_content` of each of them, labels its review outcome, and summarizes the reviewers' feedback in one sentence. The outcome is one of four labels. *Changes requested* covers threads in which reviewers accept the removal of the duplication and only ask for adjustments. *Approach questioned* covers threads in which reviewers question the deduplication itself, for instance its readability or actual benefit. *Applied* serves as a check on `commit_matches`. If a maintainer states that the patch was applied, it may have reached the tree, and the matching missed it, for example because the maintainer modified the patch. Finally, a thread is labeled *unclear* when it does not fit any of the other labels. The analysis script is available in the LKLONES repository together with the resulting CSV file.

In this sample, the threads in which reviewers questioned the deduplication itself point to a recurring reason, namely that the abstraction cost more than the duplication it removed. Reviewers said that lookup tables, macros, or shared helpers made the code less readable or added churn without a clear benefit.

VII. Limitations and Further Improvements

Threads missed by the pre-filter. The regex pre-filter only selects threads that use the terms identified in our vocabulary analysis, so discussions about duplication phrased in other ways are not identified. A future version can extend the vocabulary with terms found in the missed threads and complement the regexes with a semantic search that does not rely on exact terms.

Ambiguous or incorrect commit links. Current commit matching relies on the `Link` trailer or the patch's author and date, which may yield `AMBIGUOUS` or incorrect `EXACT` links. Likewise, a thread lacking `commit_matches` was not necessarily rejected, since its patch may have reached the mainline without a detectable link. A more robust alternative links patches to commits by change content rather than metadata. If a patch and commit leave the code in identical states, they represent the same change; thus, the code itself provides linkage evidence, regardless of who applied the patch or when. Because this approach depends neither on the `Link` trailer nor on author and date, it could reduce ambiguous and incorrect links while recovering those currently missing.

VIII. Conclusion

We introduced LKLONES, a dataset of 186,432 Linux kernel discussions on code duplication. By linking 33,209 threads to mainline commits, LKLONES captures the complete decision-making process rather than just merged refactorings, empowering empirical research on technical debt management and deduplication trade-offs in the context of Linux development.

Data and Software Availability

The dataset is publicly available on Zenodo: https://doi.org/10.5281/zenodo.23307333. The source code of LKLONES, together with its usage instructions, is available at https://gitlab.com/ccsl-usp/codev/arkanjo-research/lkml5ws-data-analysis.

Acknowledgements

The São Paulo Research Foundation – FAPESP (Proc. 2025/05395-0, 2025/22416-1) and the São Paulo State Data Analysis System Foundation – SEADE (FAPESP Proc. 2023/18026-8) partially financed this study.

REFERENCES

[1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 1st ed. Addison-Wesley, 1999.

[2] K. Kumar, S. Jarzabek, and D. Dan, "Managing big clones to ease evolution: Linux kernel example," in *Proceedings of the Federated Conference on Computer Science and Information Systems (FedCSIS)*, ser. ACSIS, vol. 8, 2016, pp. 1727–1736.

[3] W. Wang and M. W. Godfrey, "Investigating intentional clone refactoring," *Electronic Communications of the EASST*, vol. 63, 2014, proceedings of the 8th International Workshop on Software Clones (IWSC 2014).

[4] Development process: Mailing list. The kernel development community. [Online]. Available: https://www.kernel.org/doc/html/v7.3/process/2.Process.html#mailing-lists

[5] R. Passos, A. Pilone, D. Tadokoro, L. Arakaki, and P. Meirelles, "LKML5Ws: The what, when, who, where, and why in the Linux kernel mailing lists – a columnar dataset," Zenodo, 2026, dataset, version 1.0.0.

[6] C. K. Roy, J. R. Cordy, and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: A qualitative approach," *Science of Computer Programming*, vol. 74, no. 7, pp. 470–495, 2009.

[7] Submitting patches: the essential guide to getting your code into the kernel. The kernel development community. [Online]. Available: https://docs.kernel.org/process/submitting-patches.html

[8] J. Corbet. (2016) Why kernel development still uses email. Linux Weekly News. [Online]. Available: https://lwn.net/Articles/702177/

[9] Kernel.org, "lore.kernel.org." [Online]. Available: https://korg.docs.kernel.org/lore.html

[10] Y. Jiang, B. Adams, and D. M. German, "Will my patch make it? and how fast? case study on the Linux kernel," in *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*, 2013, pp. 101–110.

[11] Google Syzbot Team, "syzbot: Automated kernel bug reporting system," https://syzkaller.appspot.com, 2024, accessed: November 11, 2026.

[12] TypeSafe. (2026) Jev: TypeSafe's system one model. Version `jev-1.13.0`. [Online]. Available: https://docs.typesafe.ai/models

[13] Y. Xu and M. Zhou, "A multi-level dataset of Linux kernel patchwork," in *Proceedings of the 15th International Conference on Mining Software Repositories (MSR '18)*. ACM, 2018.